RV College of Engineering ®

Go, change the world

DEEP LEARNING

Unit – III

Sequence Modeling: Recurrent and Recursive Nets

Rajatha
Assistant Professor
Computer Science and Engineering
RV College of Engineering
Bengaluru – 59

Recurrent neural networks (RNNs)

- Processing sequential data
- Recurrent networks can scale to longer sequences
- Uses the concept-sharing parameters across different parts of a model
- Each member of the output is a function of the previous members of the output
- Each member of the output is produced using the same update rule applied to the previous outputs
- Recurrent formulation results in the sharing of parameters through a very deep computational graph

Unfolding Computational Graphs

- Example 1
- Consider a dynamical system $s^{(t)} = f(s^{(t-1)}; \theta)$
- $s(t)$ is called the state of the system
- Equation is recurrent as the definition of s at time t refers back to the same definition at time $t - 1$
- For a finite number of time steps τ , the graph can be unfolded by applying the definition $\tau - 1$ times
- For example, if we unfold equation for $\tau = 3$ time steps

$$\begin{aligned} s^{(3)} &= f(s^{(2)}; \theta) \\ &= f(f(s^{(1)}; \theta); \theta) \end{aligned}$$

Directed acyclic Graph

- Represented by a traditional directed acyclic computational graph
- The unfolded computational graph

$$s^{(t)} = f(s^{(t-1)}; \theta)$$

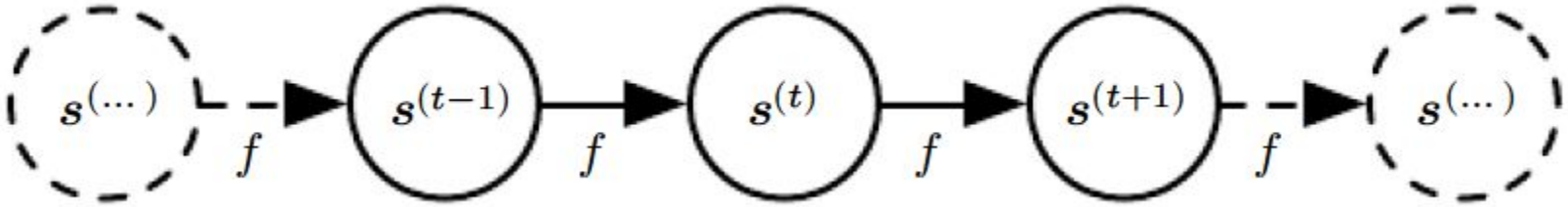


Figure 10.1: The classical dynamical system described by equation illustrated as an unfolded computational graph. Each node represents the state at some time *t* and the function *f* maps the state at *t* to the state at *t* + 1. The same parameters (the same value of Θ used to parametrize *f*) are used for all time steps.

Directed acyclic Graph

- Example2
- Consider a dynamical system driven by an external signal $x(t)$,

$$\boldsymbol{s}^{(t)} = f(\boldsymbol{s}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta})$$

- Any function involving recurrence can be considered a recurrent neural network

Directed acyclic Graph

- Hidden units are defined as

$$\boldsymbol{h}^{(t)} = f(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta}),$$

black square indicate that an interaction takes place with a delay of a single time step, from the state at time *t* to the state at time *t* + 1

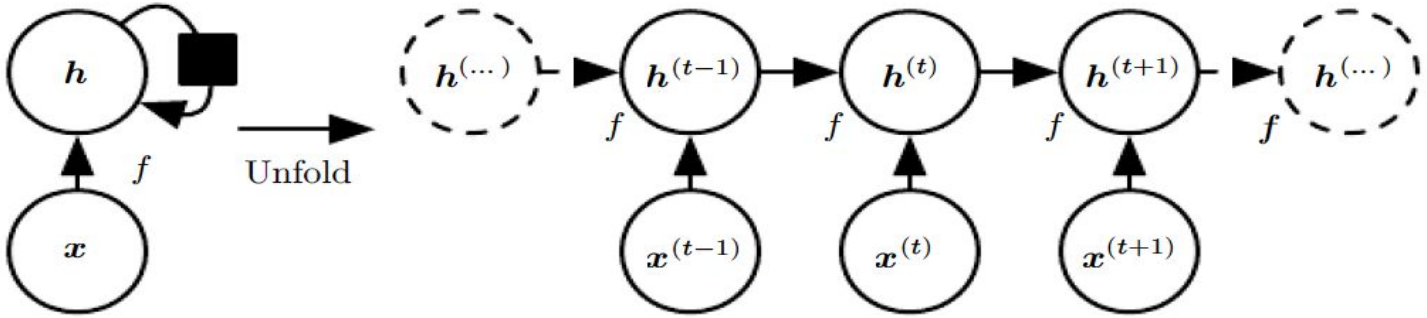


Figure 10.2: A recurrent network with no outputs. This recurrent network just processes information from the input *x* by incorporating it into the state *h* that is passed forward through time.
(Left) Circuit diagram. The black square indicates a delay of a single time step.
(Right) The same network seen as an unfolded computational graph, where each node is now associated with one particular time instance

Directed acyclic Graph

- We can represent the unfolded recurrence after t steps with $g(t)$:

$$\boldsymbol{h}^{(t)} = g^{(t)}\left(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t-1)}, \boldsymbol{x}^{(t-2)}, \ldots, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(1)}\right)$$
$$= f\left(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta}\right)$$

The function $g(t)$ takes the whole past sequence $(x(t), x(t-1), x(t-2), \ldots, x(2), x(1))$ as input and produces the current state

Unfolding process - Advantages

1. Regardless of the sequence length, the learned model always has the same input size, because it is specified in terms of transition from one state to another state, rather than specified in terms of a variable-length history of states

2. It is possible to use the *same* transition function *f* with the same parameters at every time step.

Recurrent Neural Networks

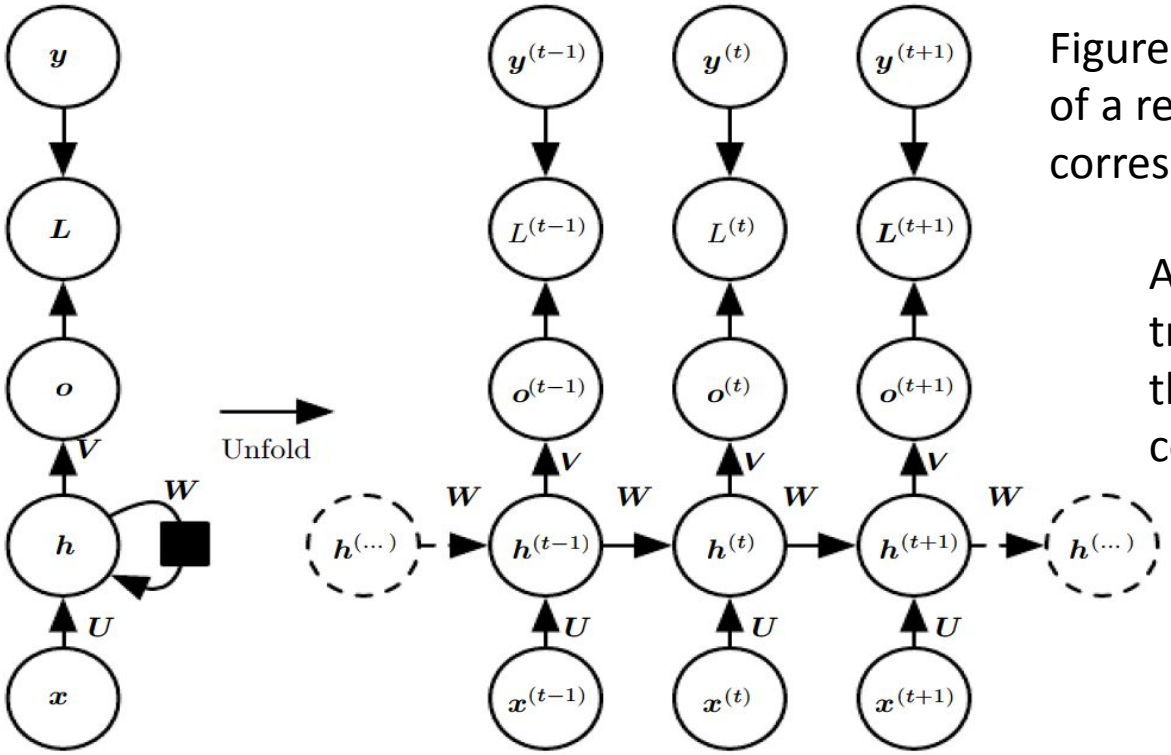


Figure 10.3: The computational graph to compute the training loss of a recurrent network that maps an input sequence of *x* values to a corresponding sequence of output *o* values.

A loss *L* measures how far each *o* is from the corresponding training target *y* . When using softmax outputs, we assume *o* is the unnormalized log probabilities. The loss *L* internally computes ˆ *y* = softmax(*o*) and compares this to the target *y*.

weight matrix *U* - input to hidden connections
weight matrix W- hidden-to-hidden recurrent connections ,
weight matrix *V*- hidden-to-output connections

*(Left)*The RNN and its loss drawn with recurrent connections.

*(Right)*The same seen as an time unfolded computational graph, where each node is now associated with one particular time instance.

Recurrent Neural Networks

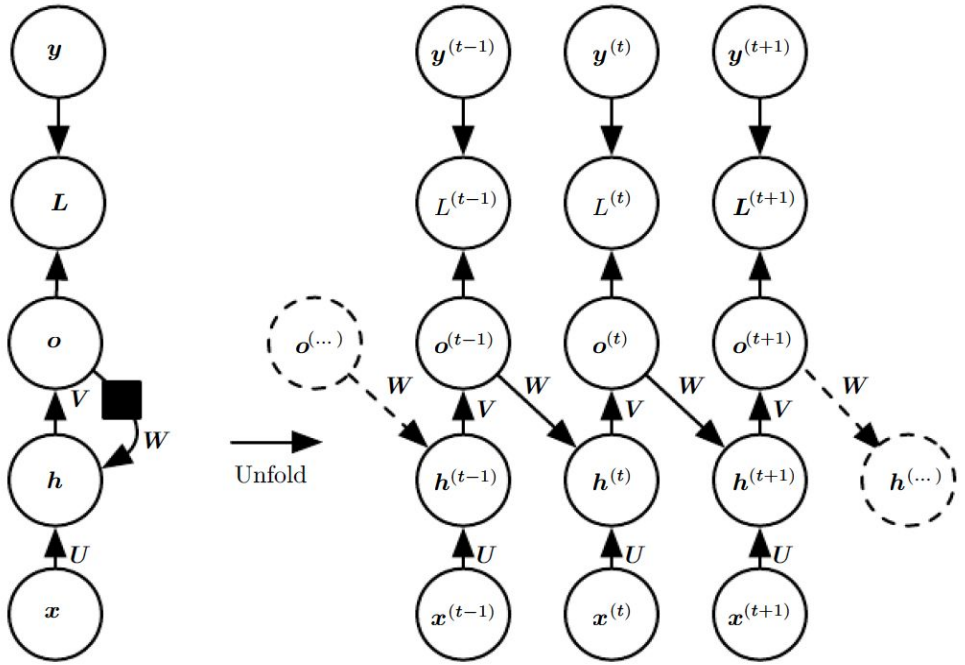
- Important design patterns for recurrent neural networks include

1. Recurrent networks that produce an output at each time step and have recurrent connections between hidden units(Figure 10.3).
2. Recurrent networks that produce an output at each time step and have recurrent connections only from the output at one time step to the hidden units at the next time step(Figure 10.4)
3. Recurrent networks with recurrent connections between hidden units, read an entire sequence and then produce a single output(Figure 10.5)

Recurrent Neural Networks

- Important design patterns for recurrent neural networks include

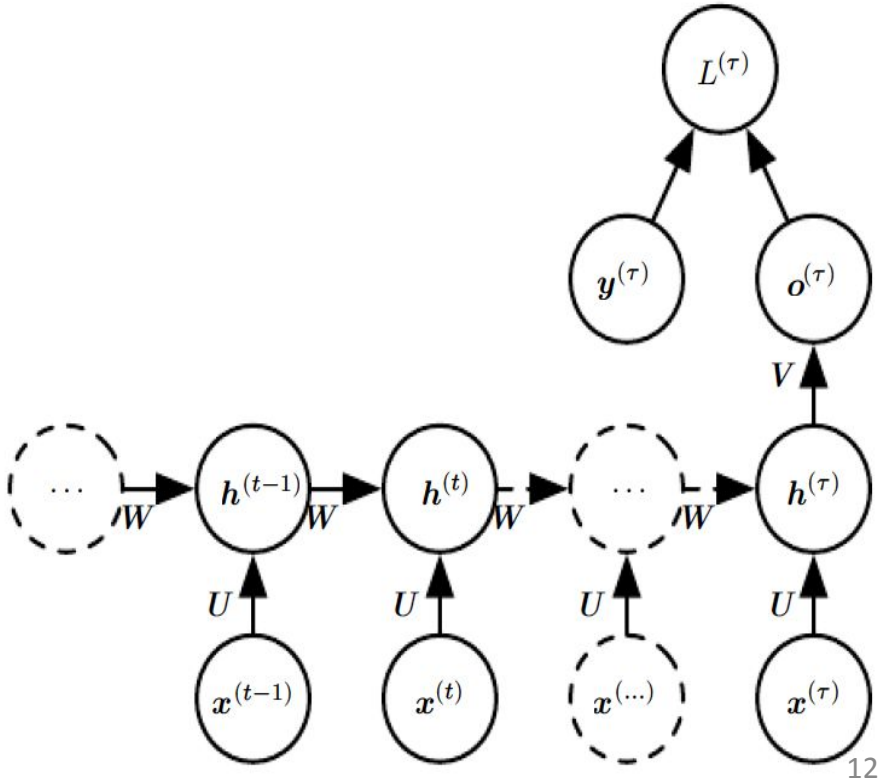
Recurrent networks that produce an output at each time step and have recurrent connections only from the output at one time step to the hidden units at the next time step(Figure 10.4)



Recurrent Neural Networks

- Important design patterns for recurrent neural networks include

Recurrent networks with recurrent connections between hidden units, read an entire sequence and then produce a single output(Figure 10.5)



Recurrent Neural Networks

- Forward propagation equations for the RNN

$t = 1$ to $t = \tau$

$$
\begin{aligned}
\boldsymbol{a}^{(t)} &= \boldsymbol{b} + \boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{U}\boldsymbol{x}^{(t)} \\
\boldsymbol{h}^{(t)} &= \tanh(\boldsymbol{a}^{(t)}) \\
\boldsymbol{o}^{(t)} &= \boldsymbol{c} + \boldsymbol{V}\boldsymbol{h}^{(t)} \\
\hat{\boldsymbol{y}}^{(t)} &= \operatorname{softmax}(\boldsymbol{o}^{(t)})
\end{aligned}
$$

- Bias vectors *b* and *c*
- *U* - input-to-hidden
- *V* - hidden-to-output
- *W* - hidden-to hidden

Recurrent Neural Networks

- Total loss for a sequence of *x* values paired with a sequence of *y* values will be sum of the losses over all the time steps
- For example, if $L^{(t)}$ is the negative log-likelihood of $y^{(t)}$ given $x^{(1)}, \ldots, x^{(t)}$ then

$$
\begin{aligned}
& L\left(\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(\tau)}\}, \{\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(\tau)}\}\right) \\
&= \sum_t L^{(t)} \\
&= -\sum_t \log p_{\text{model}}\left(y^{(t)} \mid \{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t)}\}\right),
\end{aligned}
$$

- Where $p_{\text{model}}\left(y^{(t)} \mid \{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t)}\}\right)$ is given by reading the entry for $y^{(t)}$ from the model's output vector $\hat{y}^{(t)}$

Recurrent Neural Networks

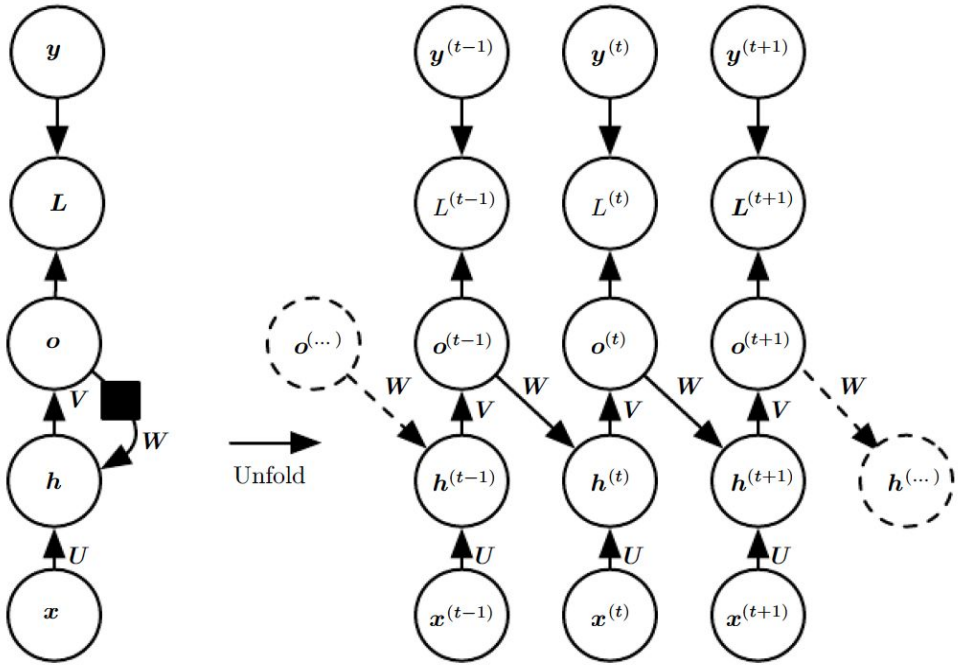


Figure 10.4: An RNN whose only recurrence is the feedback connection from the output to the hidden layer.

At each time step *t*

- Input is $x^{(t)}$
- Hidden layer activations are $h^{(t)}$
- Outputs are $o^{(t)}$
- Targets are $y^{(t)}$
- Loss is $L^{(t)}$

.

(Left) Circuit diagram

(Right) Unfolded computational graph.

- Such an RNN is less powerful (can express a smaller set of functions) than in figure 10.3.
- *o* is the only information it is allowed to send
- Easier to train because each time step can be trained in isolation from the others, allowing greater parallelization during training

Recurrent Neural Networks

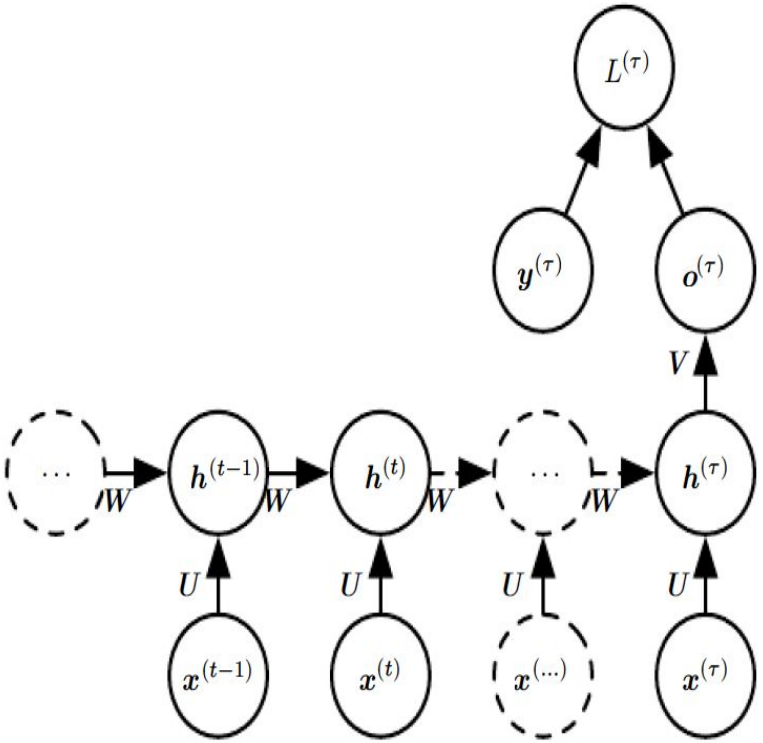


Figure 10.5: Time-unfolded recurrent neural network with a single output at the end of the sequence.

Used to summarize a sequence and produce a fixed-size representation used as input for further processing.

Teacher Forcing and Networks with Output Recurrence

- Drawback of feedback connection from the output to the hidden layer

- The network with recurrent connections only from the output at one time step to the hidden units at the next time step is less powerful
- For example, it cannot simulate a universal Turing machine Because this network lacks hidden-to-hidden recurrence, it requires that the output units capture all of the infor mation about the past that the network will use to predict the future.
- Output units are explicitly trained to match the training set targets, they are unlikely to capture the necessary information about the past history of the input, unless the user knows how to describe the full state of the system and provides it as part of the training set targets.

Teacher Forcing and Networks with Output Recurrence

- The advantage of eliminating hidden-to-hidden recurrence is that, for any loss function based on comparing the prediction at time *t* to the training target at time *t*, all the time steps are decoupled.
- Training can thus be parallelized, with the gradient for each step *t* computed in isolation.
- There is no need to compute the output for the previous time step first, because the training set provides the ideal value of that output.

Teacher Forcing and Networks with Output Recurrence

- Models that have recurrent connections from their outputs leading back into the model may be trained with **teacher forcing**.
- Teacher forcing is a procedure that emerges from the maximum likelihood criterion, in which during training, the model receives the ground truth output $y^{(t)}$ as input at time $t + 1$.
- Consider a sequence with two time steps. The conditional maximum likelihood criterion is

$$\log p\left(\boldsymbol{y}^{(1)}, \boldsymbol{y}^{(2)} \mid \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}\right)$$
$$= \log p\left(\boldsymbol{y}^{(2)} \mid \boldsymbol{y}^{(1)}, \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}\right) + \log p\left(\boldsymbol{y}^{(1)} \mid \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}\right)$$

Teacher Forcing and Networks with Output Recurrence

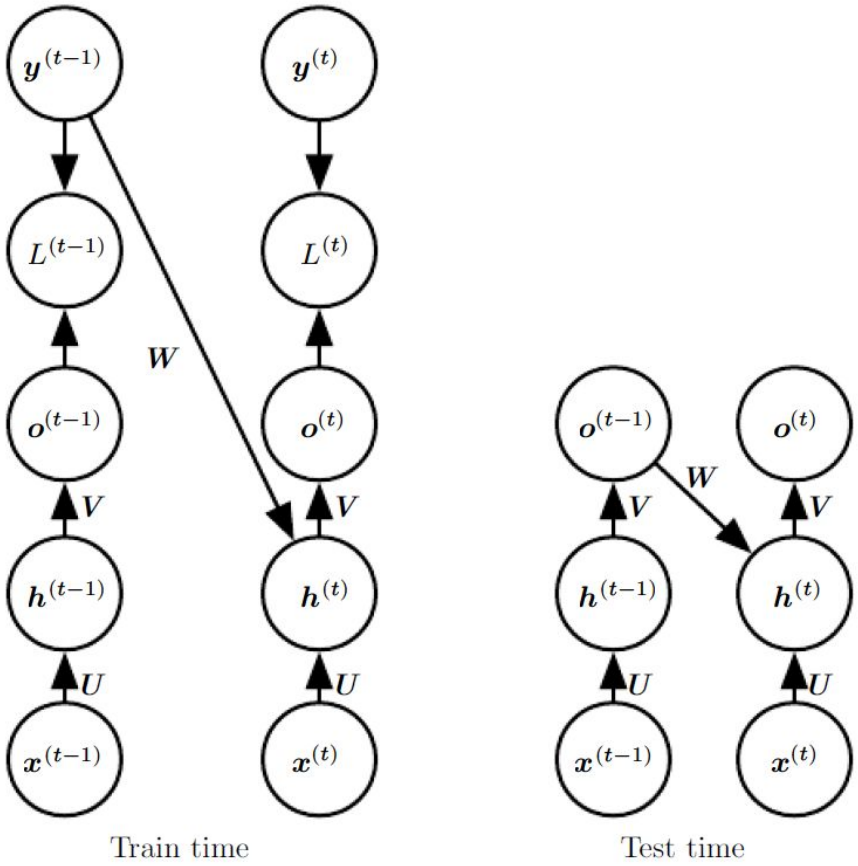


Figure 10.6: Illustration of teacher forcing

Teacher forcing is a training technique that is applicable to RNNs that have connections from their output to their hidden states at the next time step

*(Left)*At train time, we feed the *correct output* $y^{(t)}$ drawn from the train set as input to $h^{(t+1)}$

*(Right)*When the model is deployed, the true output is generally not known. In this case, we approximate the correct output $y^{(t)}$ with the model's output $o(t)$, and feed the output back into the model.

Teacher Forcing and Networks with Output Recurrence

- Teacher forcing avoid back-propagation in models that lack hidden-to-hidden connections
- Teacher forcing may still be applied to models that have hidden-to-hidden connections so long as they have connections from the output at one time step to values computed in the next time step
- If the hidden units become a function of earlier time steps, the BPTT algorithm is necessary.
- Some models may thus be trained with both teacher forcing and BPTT

Bidirectional RNNs

- Recurrent networks seen now have a "causal" structure, meaning that the state at time t only captures information from the past, $x(1), \ldots, x(t-1)$, and the present input $x(t)$.
- However, in many applications we want to output a prediction of $y(t)$ which may depend on *the whole input sequence*
- Example – Speech recognition, Handwriting recognition, Bioinformatics
- May have to look far into the future and the past to disambiguate them

Bidirectional RNNs

- Bidirectional recurrent neural networks (Bidirectional RNNs) were invented
- Combine an RNN that moves forward through time beginning from the start of the sequence with another RNN that moves backward through time beginning from the end of the sequence
- Idea can be extended to 2-dimensional input, such as images, by having *four* RNNs, each one going in one of the four directions: up, down, left, right.
- Compared to CNN, RNNs applied to images are typically more expensive

Bidirectional RNNs

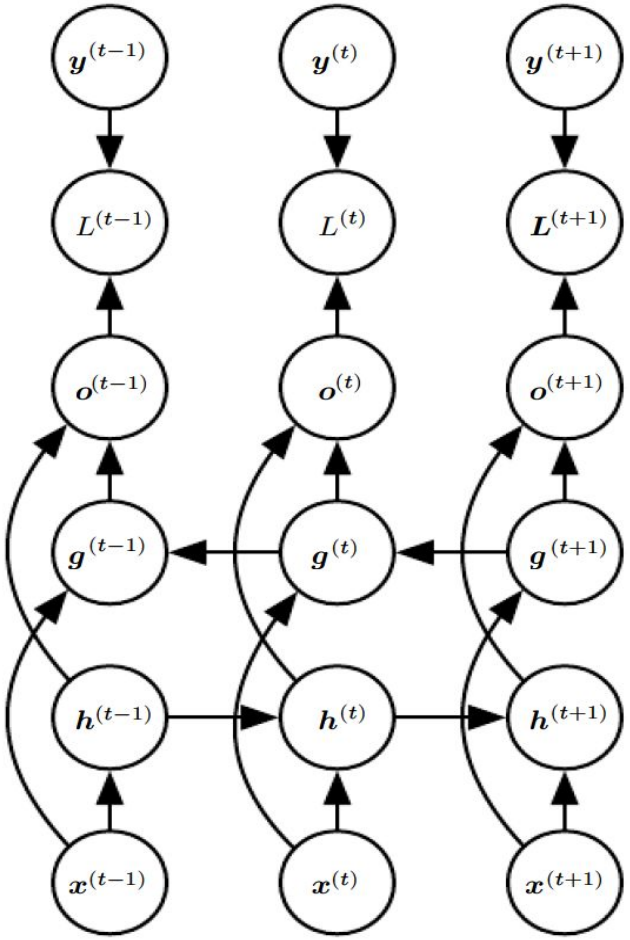


Figure 10.11: Computation of a typical bidirectional recurrent neural network

To learn to map input sequences *x* to target sequences *y*, with loss *L*(*t*) at each step *t*

- *h* recurrence propagates information forward in time (towards the right)
- *g* recurrence propagates information backward in time (towards the left)
- At each point *t*, the output units *o*(*t*) can benefit from a relevant summary of the past in its *h*(*t*) input and from a relevant summary of the future in its *g*(*t*) input

Encoder-Decoder Sequence-to-Sequence Architectures

- RNN can be trained to map an input sequence to an output sequence which is not necessarily of the same length
- Applications Speech recognition, machine translation or question answering, where the input and output sequences in the training set are generally not of the same length
- Input to the RNN Decoder is the "Context" *C*
- The context *C* might be a vector or sequence of vectors that summarize the input sequence $X = (x(1), \ldots, x(nx))$

Encoder-Decoder Sequence-to-Sequence Architectures

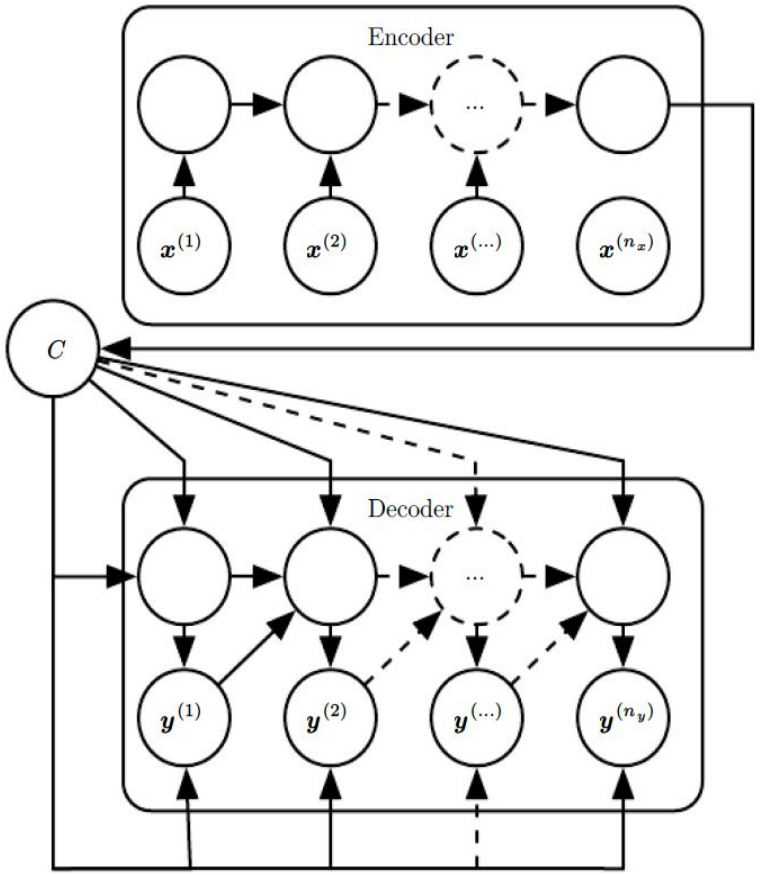


Figure 10.12: Example of an encoder-decoder or sequence-to-sequence RNN architecture, for learning to generate an output sequence (y(1), . . . , y(*n y*)) given an input sequence (x (1) , x (2), . . . , x(*nx*)).

- Encoder RNN reads the input sequence
- Decoder RNN that generates the output sequence

(or computes the probability of a given output sequence)

- The final hidden state of the encoder RNN is used to compute a generally fixed-size context variable *C* which represents a semantic summary of the input sequence and is given as input to the decoder RNN.

Encoder-Decoder Sequence-to-Sequence Architectures

- The idea behind the architecture

(1) an **encoder** or **reader** or **input** RNN processes the input sequence. The encoder emits the context C , usually as a simple function of its final hidden state.

(2) a **decoder** or **writer** or **output** RNN is conditioned on that fixed-length vector to generate the output sequence $Y = (y(1), \ldots, y(ny))$.

The innovation of this kind of architecture over those presented in earlier sections of this chapter is that the lengths nx and ny can vary from each other, while previous architectures constrained $nx = ny = \tau$.

Encoder-Decoder Sequence-to-Sequence Architectures

- If the context *C* is a vector, then the decoder RNN is simply a vector-to sequence RNN
- There are at least two ways for a vector-to-sequence RNN to receive input
  - The input can be provided as the initial state of the RNN
  - The input can be connected to the hidden units at each time step
  - Two ways can also be combined
- There is no constraint that the encoder must have the same size of hidden layer as the decoder

Deep Recurrent Networks

The computation in RNNs is decomposed into three blocks of parameters and associated transformations

1. Input to the hidden state,
2. Previous hidden state to the next hidden state
3. Hidden state to the output

Deep Recurrent Networks

- In RNN each of three blocks is associated with a single weight matrix.
- When the network is unfolded, each of these corresponds to a shallow transformation.
- Shallow transformation, mean a transformation that would be represented by a single layer within a deep MLP.
- Would it be advantageous to introduce depth in each of these operations?
- The experimental evidence shows that need enough depth in order to perform the required mappings.
- Adding depth may hurt learning by making optimization difficult
- Easier to optimize shallower architectures, and adding the extra depth of figure 10.13b makes the shortest path from a variable in time step t to a variable in time step $t + 1$ become longer.

Deep Recurrent Networks

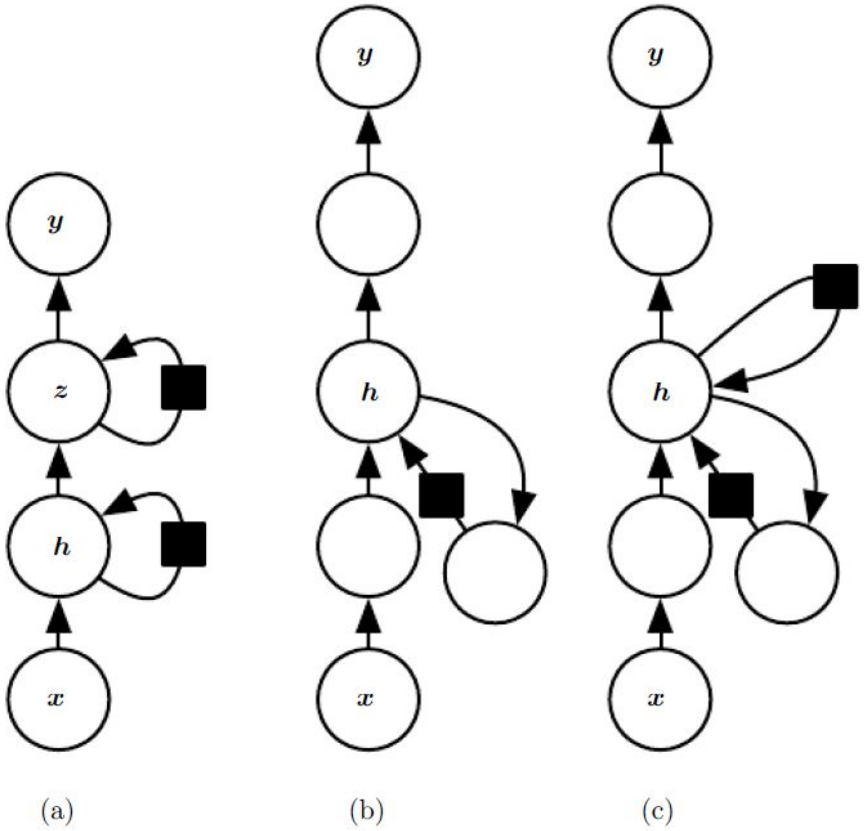


Figure 10.13: A recurrent neural network can be made deep in many ways
The hidden recurrent state
(a) can be broken down into groups organized hierarchically.
*(b)*Deeper computation (e.g., an MLP) can be introduced in the input-to hidden, hidden-to-hidden and hidden-to-output parts. This may lengthen the shortest path linking different time steps.
*(c)*The path-lengthening effect can be mitigated by introducing skip connections.

Recursive Neural Networks

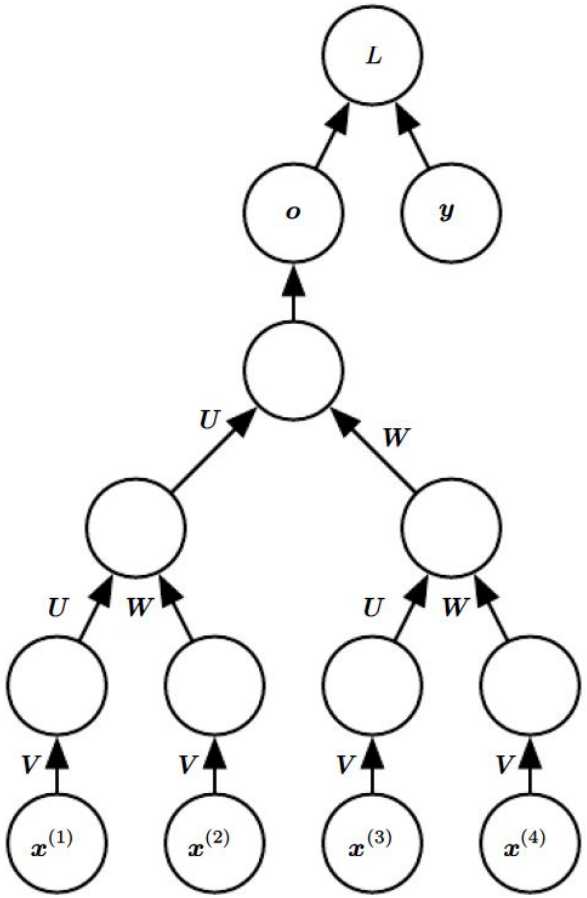


Figure 10.14: A recursive network has a computational graph that generalizes that of the recurrent network from a chain to a tree.

A variable-size sequence $x(1), x(2), \ldots, x(t)$ can be mapped to a fixed-size representation (the output o), with a fixed set of parameters (the weight matrices U, V, W).

Recursive Neural Networks

- Recursive neural networks represent generalization of recurrent networks, with a different kind of computational graph, which is structured as a deep tree, rather than the chain-like structure of RNNs.
- Recursive networks have been successfully applied to processing *data structures* as input to neural nets in natural language processing as well as in computer vision
- Advantage of recursive nets over recurrent nets is that for a sequence of the same length τ, the depth can be drastically reduced from τ to $O(\log \tau)$, which might help deal with long-term dependencies.

Recursive Neural Networks

- Many variants of the recursive net

Associate the data with a tree structure, and associate the inputs and targets with individual nodes of the tree.

- The computation performed by each node does not have to be the traditional artificial neuron computation (affine transformation of all inputs followed by a monotone nonlinearity).
- Can use tensor operations and bilinear forms

The Long Short-Term Memory and Other Gated RNNs

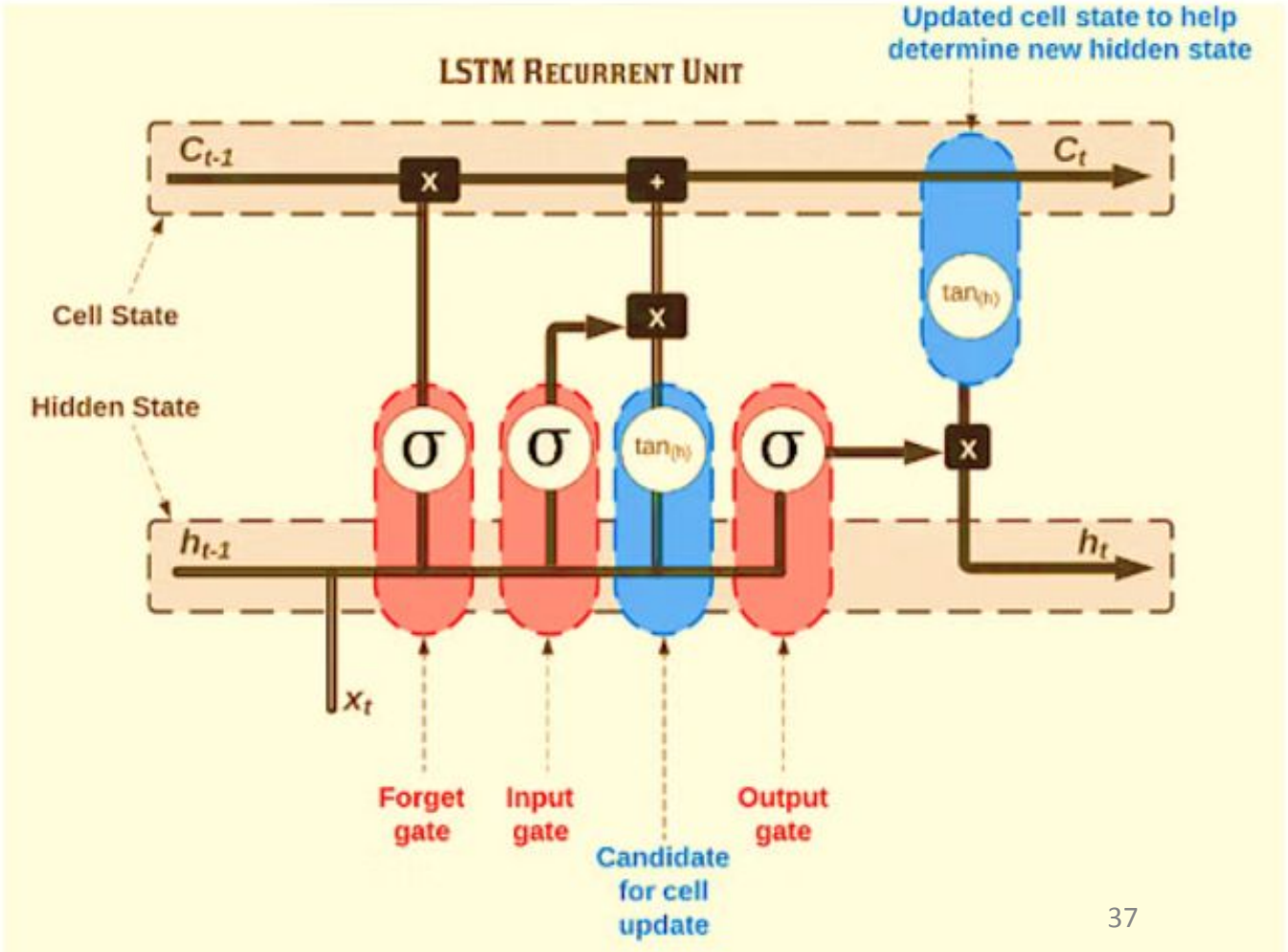
- Most effective sequence models used in practical applications are called **gated RNNs- Example long short-term memory**
- Networks are based on the **gated recurrent unit**
- Like leaky units, gated RNNs are based on the idea of creating paths through time that have derivatives that neither vanish nor explode.
- Leaky units did this with connection weights that were either manually chosen constants or were parameters.
- Gated RNNs generalize this to connection weights that may change at each time step

The Long Short-Term Memory and Other Gated RNNs

- Leaky units allow the network to *accumulate* information (such as evidence for a particular feature or category) over a long duration.
- However, once that information has been used, it might be useful for the neural network to *forget* the old state.
- For example, if a sequence is made of sub-sequences and we want a leaky unit to accumulate evidence inside each sub-subsequence, we need a mechanism to forget the old state by setting it to zero.
- Instead of manually deciding when to clear the state, we want the neural network to learn to decide when to do it. This is what gated RNNs do.

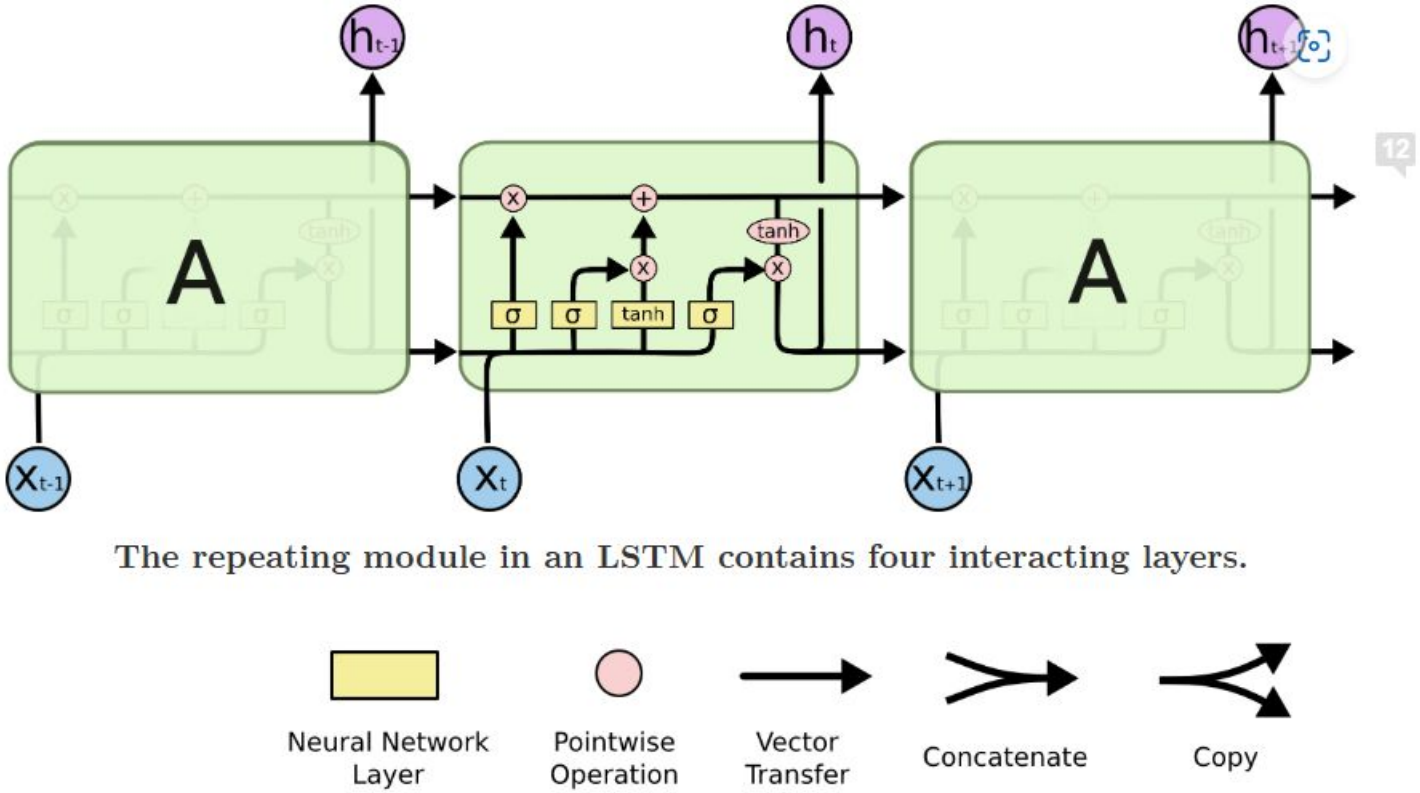
LSTM - Long Short-Term Memory

- The structure of an LSTM network consists of a series of LSTM cells,
- Each cell has a set of gates (input, output, and forget gates) that control the flow of information into and out of the cell.
- The gates are used to selectively forget or retain information from the previous time steps, allowing the LSTM to maintain long-term dependencies in the input data.



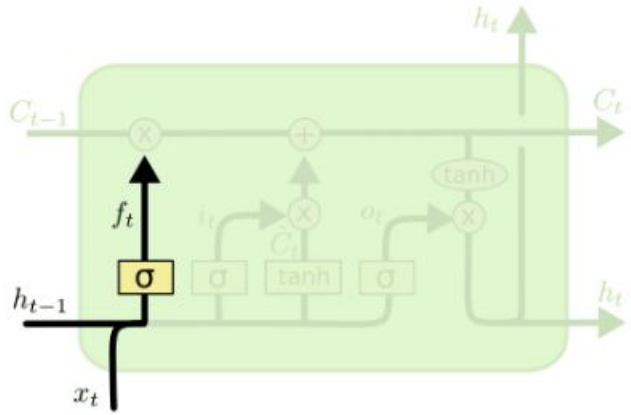
- The cell state is kind of like a conveyor belt. It runs straight down the entire chain, with only some minor linear interactions.

- The LSTM does have the ability to remove or add information to the cell state, carefully regulated by structures called gates.

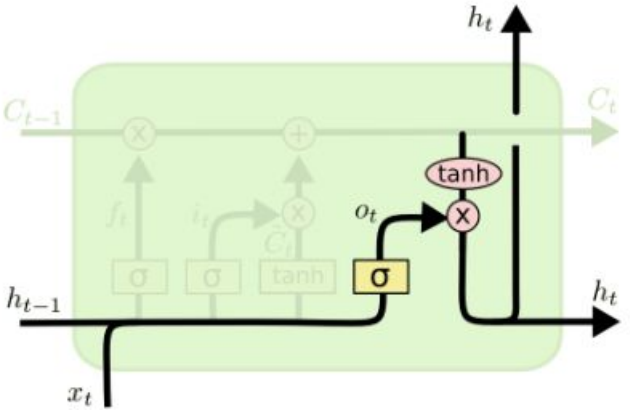


The repeating module in an LSTM contains four interacting layers.

Step-by-Step LSTM Walk Through

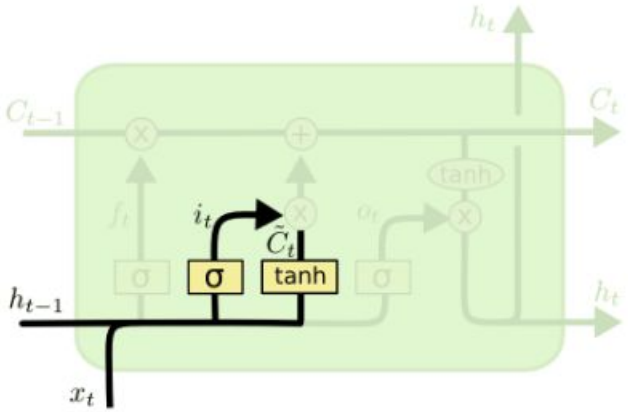


$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$



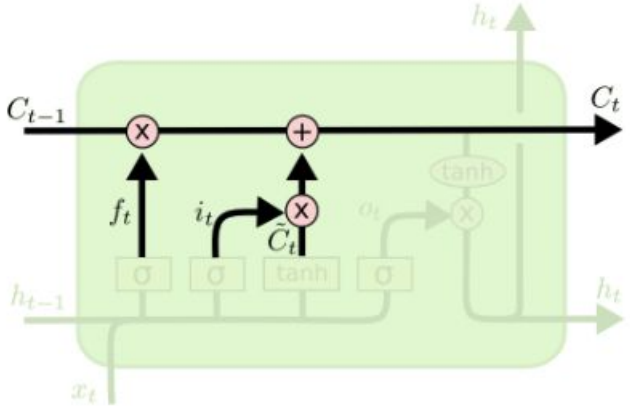
$$o_t = \sigma\left(W_o \; [h_{t-1}, x_t] \; + \; b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM

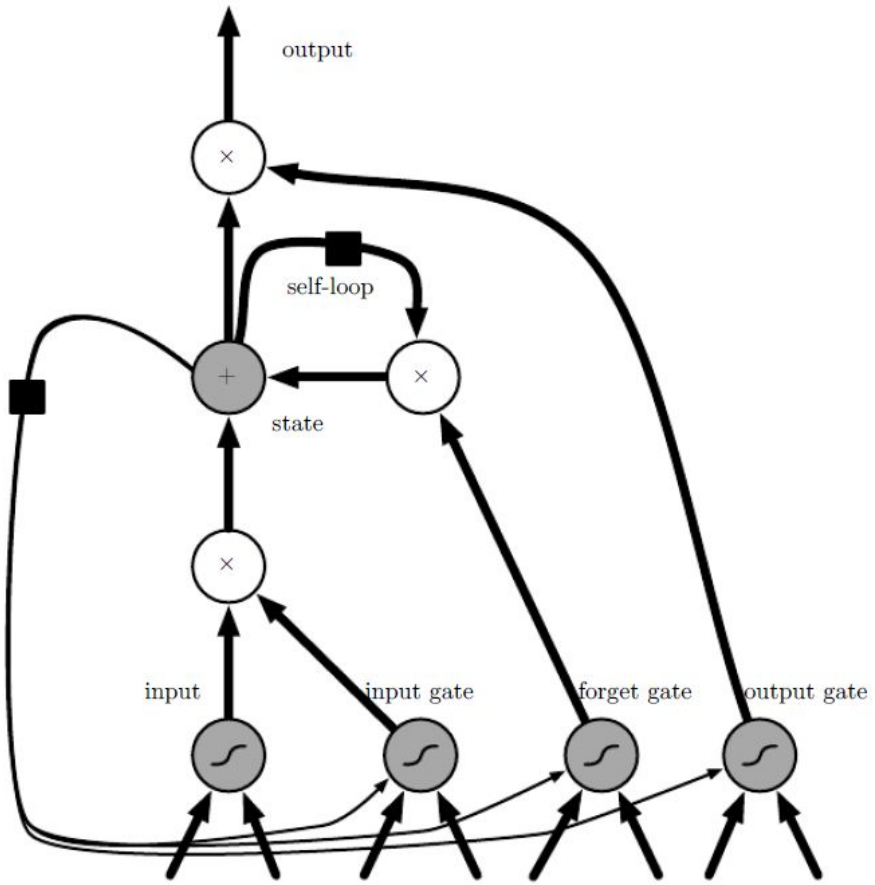


Figure 10.16: Block diagram of the LSTM recurrent network “cell.”

- Cells are connected recurrently to each other, replacing the usual hidden units of ordinary recurrent networks.
- An input feature is computed with a regular artificial neuron unit.

Its value can be accumulated into the state if the sigmoidal input gate allows it.

- The state unit has a linear self-loop whose weight is controlled by the forget gate.
- The output of the cell can be shut off by the output gate.
- All the gating units have a sigmoid nonlinearity, while the input unit can have any squashing nonlinearity.
- The state unit can also be used as an extra input to the gating units.
- The black square indicates a delay of a single time step.

LSTM

- The idea of introducing self-loops to produce paths where the gradient can flow for long durations is a core contribution of the initial **long short-term memory (LSTM)** model.
- A crucial addition has been to make the weight on this self-loop conditioned on the context, rather than fixed. By making the weight of this self-loop gated (controlled by another hidden unit), the time scale of integration can be changed dynamically.
- In this case, we mean that even for an LSTM with fixed parameters, the time scale of integration can change based on the input sequence, because the time constants are output by the model itself.
- The LSTM has been found extremely successful in many applications, such as unconstrained handwriting recognition, speech recognition, handwriting generation, machine translation, image captioning and parsing

LSTM

- LSTM recurrent networks have "LSTM cells" that have an internal recurrence (a self-loop), in addition to the outer recurrence of the RNN.
- Each cell has the same inputs and outputs as an ordinary recurrent network, but has more parameters and a system of gating units that controls the flow of information.
- The most important component is the state unit $s(t)i$ that has a linear self-loop similar to the leaky units described in the previous section. However, here, the self-loop weight (or the associated time constant) is controlled by a **forget gate** unit $f(t)$ i (for time step t and cell i), that sets this weight to a value between 0 and 1 via a sigmoid unit:

LSTM-Forget Gate

$$f_i^{(t)} = \sigma\left(b_i^f + \sum_j U_{i,j}^f x_j^{(t)} + \sum_j W_{i,j}^f h_j^{(t-1)}\right)$$

where $x(t)$ is the current input vector and $h(t)$ is the current hidden layer vector, containing the outputs of all the LSTM cells, and bf ,Uf , Wf are respectively biases, input weights and recurrent weights for the forget gates.

LSTM-Input Gate

- The LSTM cell internal state is thus updated as follows, but with a conditional self-loop weight $f(t)i$:

$$s_i^{(t)} = f_i^{(t)} s_i^{(t-1)} + g_i^{(t)} \sigma \left(b_i + \sum_j U_{i,j} x_j^{(t)} + \sum_j W_{i,j} h_j^{(t-1)} \right)$$

- where b, U and W respectively denote the biases, input weights and recurrent weights into the LSTM cell.
- The **external input gate** unit $g(t)$ i is computed similarly to the forget gate (with a sigmoid unit to obtain a gating value between 0 and 1), but with its own parameters:

$$g_i^{(t)} = \sigma \left(b_i^g + \sum_j U_{i,j}^g x_j^{(t)} + \sum_j W_{i,j}^g h_j^{(t-1)} \right)$$

LSTM

- The output $h(t)$ *i* of the LSTM cell can also be shut off, via the **output gate** q $(t)i$, which also uses a sigmoid unit for gating:

$$h_i^{(t)} = \tanh\left(s_i^{(t)}\right) q_i^{(t)}$$

$$q_i^{(t)} = \sigma\left(b_i^o + \sum_j U_{i,j}^o x_j^{(t)} + \sum_j W_{i,j}^o h_j^{(t-1)}\right)$$

Parameters *bo*, *Uo*, *Wo* for its biases, input weights and recurrent weights

Other Gated RNNs

- Which pieces of the LSTM architecture are actually necessary? What other successful architectures could be designed that allow the network to dynamically control the time scale and forgetting behavior of different units?
- Answers

Gated recurrent units or GRUs

- The main difference with the LSTM is that a single gating unit simultaneously controls the forgetting factor and the decision to update the state unit. The update equations are the following:

$$h_i^{(t)} = u_i^{(t-1)} h_i^{(t-1)} + (1 - u_i^{(t-1)}) \sigma \left(b_i + \sum_j U_{i,j} x_j^{(t-1)} + \sum_j W_{i,j} r_j^{(t-1)} h_j^{(t-1)} \right)$$

Other Gated RNNs

- where *u* stands for "update" gate and *r* for "reset" gate. Their value is defined as usual:

$$u_i^{(t)} = \sigma\left(b_i^u + \sum_j U_{i,j}^u x_j^{(t)} + \sum_j W_{i,j}^u h_j^{(t)}\right)$$

$$r_i^{(t)} = \sigma\left(b_i^r + \sum_j U_{i,j}^r x_j^{(t)} + \sum_j W_{i,j}^r h_j^{(t)}\right).$$

Other Gated RNNs

- Reset and updates gates can individually “ignore” parts of the state vector
- The update gates act like conditional leaky integrators that can linearly gate any dimension, thus choosing to copy it or completely ignore it by replacing it by the new “target state” value
- The reset gates control which parts of the state get used to compute the next target state, introducing an additional nonlinear effect in the relationship between past state and future state.

Echo State Networks

- The recurrent weights mapping from $h(t-1)$ to $h(t)$ and the input weights mapping from $x(t)$ to $h(t)$ are some of the most difficult parameters to learn in a recurrent network.
- Proposed approach to avoiding this difficulty

Echo state networks or ESNs-- set the recurrent weights such that the recurrent hidden units do a good job of capturing the history of past inputs, and *learn only the output weights*.

Liquid state machines—Similar to ESN, except that it uses spiking neurons (with binary outputs) instead of the continuous-valued hidden units used for ESNs.

Both ESNs and liquid state machines are termed **reservoir computing**

- **reservoir computing** the hidden units form of reservoir of temporal features which may capture different aspects of the history of inputs.

Echo State Networks

- The important question is therefore: how do we set the input and recurrent weights so that a rich set of histories can be represented in the recurrent neural network state?
- View the recurrent net as a dynamical system, and set the input and recurrent weights such that the dynamical system is near the edge of stability

Echo State Networks

- The original idea was to make the eigenvalues of the Jacobian of the state-to statetransition function be close to 1.

An important characteristic of a recurrent network is the eigenvalue spectrum of the JacobiansOf particular importance is the **spectral radius** of $J(t)$, defined to be the maximum of the absolute values of its eigenvalues.

$$J(t) = \partial s(t) \partial s(t-1)$$

Echo State Networks

- When a linear map W □ always shrinks h as measured by the $L2$ norm, then we say that the map is **contractive**.
- When the spectral radius is less than one, the mapping from $h(t)$ to $h(t+1)$ is contractive, so a small change becomes smaller after each time step.
- This necessarily makes the network forget information about the past when we use a finite level of precision to store the state vector

Echo State Networks

- The Jacobian matrix tells us how a small change of $h(t)$ propagates one step forward, or equivalently, how the gradient on $h(t+1)$ propagates one step backward,during back-propagation
- Neither W nor J need to be symmetric so they can have complex-valued eigenvalues and eigenvectors, with imaginary components corresponding to potentially oscillatory behavior

Computing the Gradient in a Recurrent Neural Network

- Computing the gradient in RNN is straightforward.
- One simply applies the generalized back-propagation algorithm to the unrolled computational graph.
- No specialized algorithms are necessary.
- Gradients obtained by back-propagation may then be used with any general-purpose gradient-based techniques to train an RNN.

Computing the Gradient in a Recurrent Neural Network

- To gain some intuition for how the BPTT algorithm behaves, we provide an example of how to compute gradients by BPTT for the RNN equations above (equation 10.8 and equation 10.12).
- The nodes of our computational graph include the parameters U , V , W , b and c as well as the sequence of nodes indexed by t for $x(t)$, $h(t)$, $o(t)$ and $L(t)$.
- For each node **N** we need to compute the gradient $\nabla_{\mathbf{N}} L$ recursively, based on the gradient computed at nodes that follow it in the graph. We start the recursion with the nodes immediately preceding the final loss

$$\frac{\partial L}{\partial L^{(t)}} = 1$$

Computing the Gradient in a Recurrent Neural Network

- In this derivation we assume that the outputs $o(t)$ are used as the argument to the softmax function to obtain the vector $\hat{y}$ of probabilities over the output. We also assume that the loss is the negative log-likelihood of the true target $y(t)$ given the input so far. The gradient $\nabla o(t) L$ on the outputs at time step t, for all i, t, is as follows:

$$(\nabla_{\boldsymbol{o}^{(t)}} L)_i = \frac{\partial L}{\partial o_i^{(t)}} = \frac{\partial L}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o_i^{(t)}} = \hat{y}_i^{(t)} - \mathbf{1}_{i,y^{(t)}}$$

Computing the Gradient in a Recurrent Neural Network

- We work our way backwards, starting from the end of the sequence. At the final time step τ , $h(\tau)$ only has $o(\tau)$ as a descendent, so its gradient is simple:

$$\nabla_{\boldsymbol{h}(\tau)} L = \boldsymbol{V}^\top \nabla_{\boldsymbol{o}(\tau)} L.$$

Computing the Gradient in a Recurrent Neural Network

- We can then iterate backwards in time to back-propagate gradients through time, from $t = \tau - 1$ down to $t = 1$, noting that $h(t)$ (for $t < \tau$) has as descendants both $o(t)$ and $h(t+1)$. Its gradient is thus given by

$$\nabla_{\boldsymbol{h}^{(t)}} L = \left(\frac{\partial \boldsymbol{h}^{(t+1)}}{\partial \boldsymbol{h}^{(t)}} \right)^{\top} (\nabla_{\boldsymbol{h}^{(t+1)}} L) + \left(\frac{\partial \boldsymbol{o}^{(t)}}{\partial \boldsymbol{h}^{(t)}} \right)^{\top} (\nabla_{\boldsymbol{o}^{(t)}} L)$$

$$= \boldsymbol{W}^{\top} (\nabla_{\boldsymbol{h}^{(t+1)}} L) \operatorname{diag} \left(1 - \left(\boldsymbol{h}^{(t+1)} \right)^2 \right) + \boldsymbol{V}^{\top} (\nabla_{\boldsymbol{o}^{(t)}} L)$$

Computing the Gradient in a Recurrent Neural Network

- Diag indicates the diagonal matrix containing the elements
  This is the Jacobian of the hyperbolic tangent associated with the hidden unit *i* at time *t* + 1.

$$1 - (h_i^{(t+1)})^2$$

Computing the Gradient in a Recurrent Neural Network

- Using this notation, the gradient on the remaining parameters is given by:

$$\nabla_{\boldsymbol{c}} L = \sum_t \left(\frac{\partial \boldsymbol{o}^{(t)}}{\partial \boldsymbol{c}} \right)^\top \nabla_{\boldsymbol{o}^{(t)}} L = \sum_t \nabla_{\boldsymbol{o}^{(t)}} L$$

$$\nabla_{\boldsymbol{b}} L = \sum_t \left(\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{b}^{(t)}} \right)^\top \nabla_{\boldsymbol{h}^{(t)}} L = \sum_t \operatorname{diag}\left(1 - \left(\boldsymbol{h}^{(t)} \right)^2 \right) \nabla_{\boldsymbol{h}^{(t)}} L$$

$$\nabla_{\boldsymbol{V}} L = \sum_t \sum_i \left(\frac{\partial L}{\partial o_i^{(t)}} \right) \nabla_{\boldsymbol{V}} o_i^{(t)} = \sum_t \left(\nabla_{\boldsymbol{o}^{(t)}} L \right) \boldsymbol{h}^{(t)\top}$$

$$\nabla_{\boldsymbol{W}} L = \sum_t \sum_i \left(\frac{\partial L}{\partial h_i^{(t)}} \right) \nabla_{\boldsymbol{W}^{(t)}} h_i^{(t)}$$

$$= \sum_t \operatorname{diag}\left(1 - \left(\boldsymbol{h}^{(t)} \right)^2 \right) \left(\nabla_{\boldsymbol{h}^{(t)}} L \right) \boldsymbol{h}^{(t-1)\top}$$

Computing the Gradient in a Recurrent Neural Network

$$\begin{aligned}\nabla_{\boldsymbol{U}} L &= \sum_t \sum_i \left(\frac{\partial L}{\partial h_i^{(t)}} \right) \nabla_{\boldsymbol{U}^{(t)}} h_i^{(t)} \\ &= \sum_t \operatorname{diag}\left(1 - \left(\boldsymbol{h}^{(t)} \right)^2 \right) \left(\nabla_{\boldsymbol{h}^{(t)}} L \right) \boldsymbol{x}^{(t)\top}\end{aligned}$$

We do not need to compute the gradient with respect to *x*(*t*) for training because it does not have any parameters as ancestors in the computational graph defining the loss.

Recurrent Networks as Directed Graphical Models

- Losses $L(t)$ were cross-entropies between training targets $y(t)$ and outputs $o(t)$
- The loss should be chosen based on the task.
- Usually interpret the output of the RNN as a probability distribution
- we usually use the cross-entropy associated with that distribution to define the loss
- Mean squared error is the cross-entropy loss associated with an output distribution that is a unit Gaussian

Recurrent Networks as Directed Graphical Models

- When we use a predictive log-likelihood training objective, we train the RNN to estimate the conditional distribution of the next sequence element *y*(*t*) given the past inputs. This may mean that we maximize the log-likelihood

$$\log p(\boldsymbol{y}^{(t)} \mid \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t)}),$$

- or, if the model includes connections from the output at one time step to the next time step,

$$\log p(\boldsymbol{y}^{(t)} \mid \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t)}, \boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(t-1)})$$

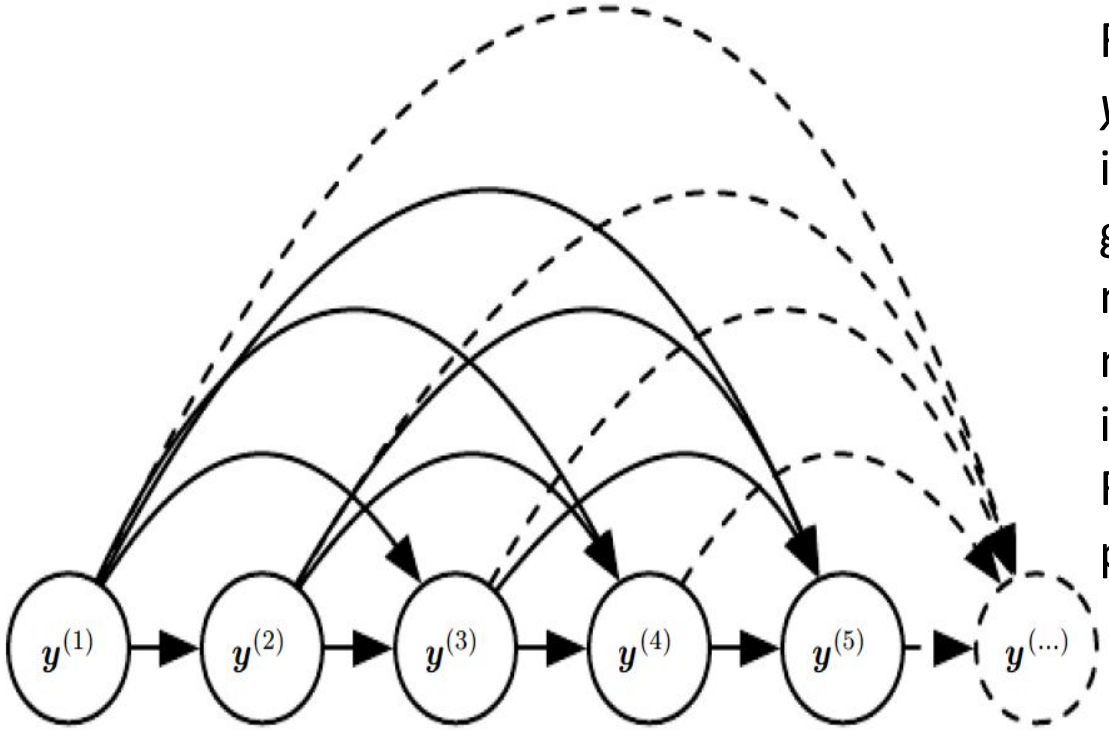


Figure 10.7: Fully connected graphical model for a sequence $y(1), y(2), \ldots, y(t), \ldots$: every past observation $y(i)$ may influence the conditional distribution of some $y(t)$ (for $t > i$), given the previous values. Parametrizing the graphical model directly according to this graph (as in equation 10.6) might be very inefficient, with an ever growing number of inputs and parameters for each element of the sequence. RNNs obtain the same full connectivity but efficient parametrization, as illustrated in figure 10.8.

$$L = \sum_t L^{(t)}$$

$$L^{(t)} = -\log P(\mathrm{y}^{(t)} = y^{(t)} \mid y^{(t-1)}, y^{(t-2)}, \ldots, y^{(1)}).$$

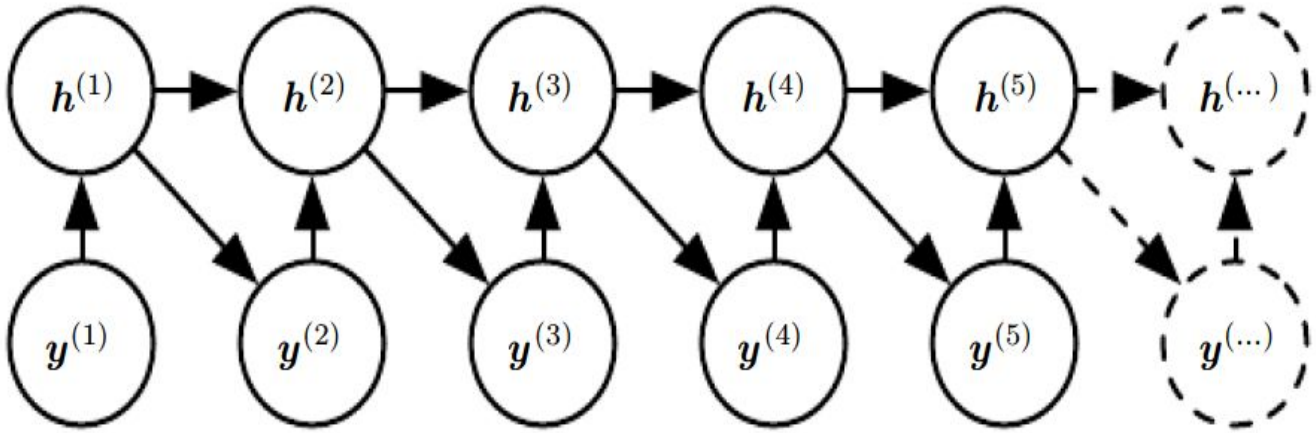


Figure 10.8: Introducing the state variable in the graphical model of the RNN, even though it is a deterministic function of its inputs, helps to see how we can obtain a very efficient parametrization, based on equation 10.5. Every stage in the sequence (for $h(t)$ and $y(t)$) involves the same structure (the same number of inputs for each node) and can share the same parameters with the other stages.

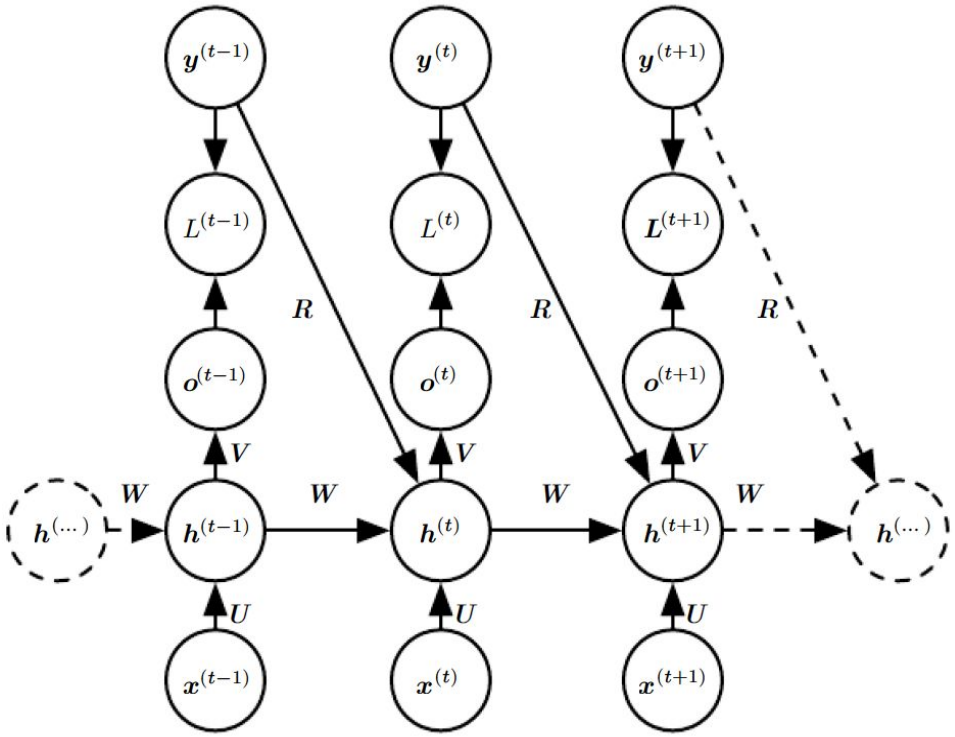


Figure 10.10: A conditional recurrent neural network mapping a variable-length sequence of *x* values into a distribution over sequences of *y* values of the same length. Compared to figure 10.3, this RNN contains connections from the previous output to the current state. These connections allow this RNN to model an arbitrary distribution over sequences of *y* given sequences of *x* of the same length. The RNN of figure 10.3 is only able to represent distributions in which the *y* values are conditionally independent from each other given the *x* values.

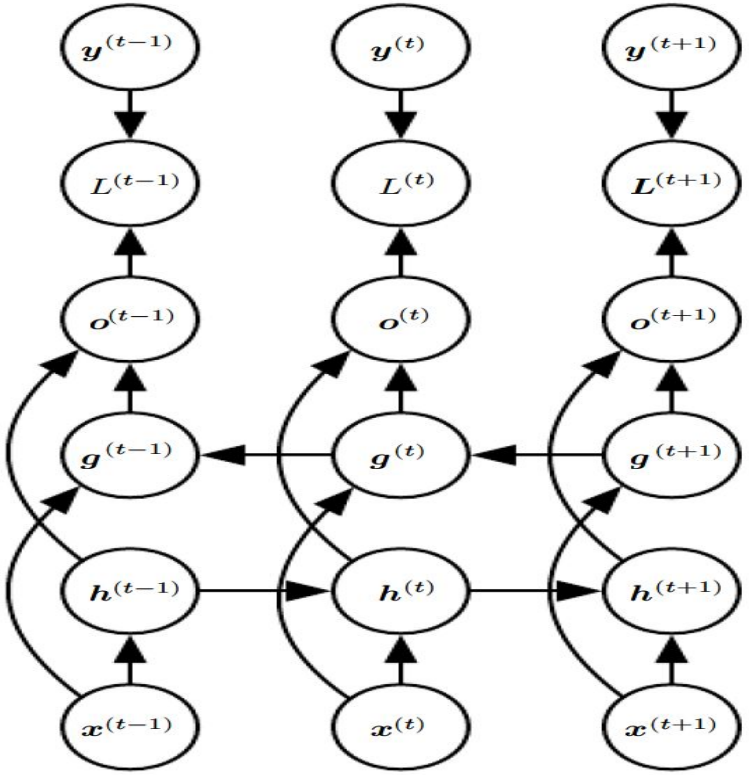


Figure 10.11: Computation of a typical bidirectional recurrent neural network, meant to learn to map input sequences *x* to target sequences *y*, with loss *L*(*t*) at each step *t*. The *h* recurrence propagates information forward in time (towards the right) while the *g* recurrence propagates information backward in time (towards the left). Thus at each point *t*, the output units *o*(*t*) can benefit from a relevant summary of the past in its *h*(*t*) input and from a relevant summary of the future in its *g*(*t*) input.

Thank You